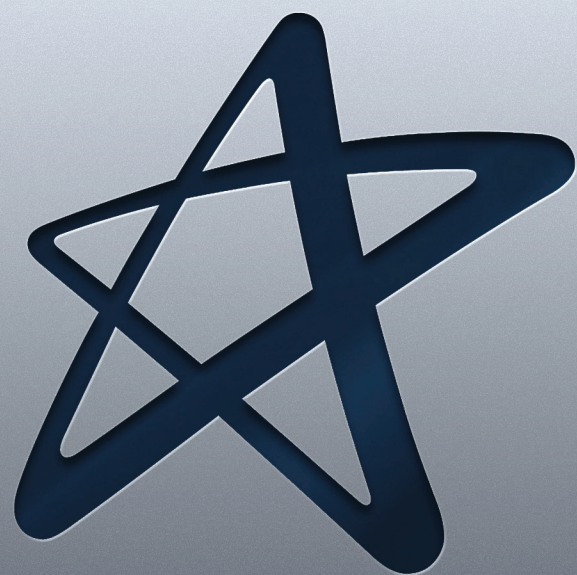


Estruturas de Dados Lineares



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico



Algoritmos de Classificação e Pesquisa I

Responsável pelo Conteúdo:

Prof. Dr. Luciano Rossi

Revisão Textual:

Prof.^a Esp. Kelciane de Rocha Campos

UNIDADE

Algoritmos de Classificação e Pesquisa I



- Introdução;
- Ordenação pelo Método de Bolhas (*Bubble Sort*);
- Ordenação por Inserção (*Insertion Sort*);
- Ordenação por Seleção (*Selection Sort*);
- Pesquisa Sequencial.



OBJETIVOS DE APRENDIZADO

- Apresentar ao aluno os algoritmos de ordenação (classificação) fundamentais e suas respectivas implementações iterativas;
- Despertar, de maneira empírica, o olhar do aluno para as diferenças entre os algoritmos de ordenação quanto a estratégia e desempenho computacional;
- Introduzir o conceito de pesquisa (busca) em estruturas estáticas, avaliando suas limitações.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu “momento do estudo”;
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e sites para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

Nesta unidade, teremos a oportunidade de estudar diferentes métodos de ordenação ou classificação. Na computação, esses métodos despertam um especial interesse, pois a tarefa de ordenar conjuntos pode ser feita de várias maneiras diferentes, cada qual com uma especificidade que enriquece o estudo das diferentes estratégias consideradas, elucidando novas possibilidades, que, certamente, serão úteis em sua atuação acadêmica e profissional.

Outro assunto que será abordado nesta unidade é a pesquisa ou busca sequencial em listas de elementos. Identificar um determinado elemento, que está inserido em uma lista de elementos, é uma tarefa cotidiana para os profissionais da área da computação. Apesar de triviais, as tarefas de ordenação e de busca podem nos revelar algumas características que são particularmente interessantes para o desenvolvimento de aplicações computacionais.

Podemos observar vários exemplos de aplicação da ordenação e da busca em programas que utilizamos em nosso dia a dia. As planilhas eletrônicas disponibilizam essas funcionalidades, podemos ordenar uma lista de registros com um simples clicar do mouse. Da mesma forma, os chamados Sistemas Gerenciadores de Banco de Dados (SGBD) executam essas tarefas sem qualquer esforço por parte do usuário do sistema. Porém, é importante ter em mente que há um algoritmo executando essas tarefas e que ele pode ter características que o tornam mais ou menos eficiente.

Conhecer e utilizar as funcionalidades disponibilizadas pelos algoritmos não implica saber, de fato, como elas executam as tarefas. Os algoritmos de ordenação e de busca foram implementados nesses programas de modo que seja possível a simples utilização por parte do usuário. No contexto deste curso, estamos interessados em como os algoritmos funcionam, permitindo, assim, que migremos da condição de usuário para uma condição superior, na qual poderemos aplicar os conhecimentos adquiridos em outros desafios que possam surgir.

Vamos iniciar nossos estudos com os algoritmos de ordenação. Primeiro, é preciso definir, formalmente, o que é o problema de ordenação. Como em diversos outros casos na computação, o problema de ordenação consiste em transformar uma entrada em uma saída, ambas bem especificadas. Desse modo, o algoritmo que resolve esse problema deve descrever os passos necessários para que a entrada seja transformada na saída que pretendemos. Assim, Cormen (2009) define a entrada e a saída para um problema de ordenação da seguinte forma:

- **Entrada:** uma sequência de n números $[x_1, x_2, \dots, x_n]$;
- **Saída:** uma permutação da entrada $[x'_1, x'_2, \dots, x'_n]$ tal que $x'_1 \leq x'_2 \leq \dots \leq x'_n$.

Veja que as descrições da entrada e da saída são genéricas, onde cada x representa um valor na sequência. Especificamente para a saída, um valor anterior deve ser menor ou igual ao valor posterior. Por exemplo, considere a sequência $[3, 5, 1, 8, 4, 0]$, a qual é a entrada do nosso problema de ordenação. Assim, a saída correspondente seria $[0, 1, 3, 4, 5, 8]$.

O problema de pesquisa ou busca pode ser definido da seguinte forma:

- **Entrada:** uma sequência de n números $V = [x_1, x_2, \dots, x_n]$ e um valor y a ser pesquisado;
- **Saída:** o valor lógico verdadeiro, caso $y \in V$, ou falso, caso contrário.

As próximas seções serão dedicadas à descrição de alguns algoritmos de ordenação. Esses algoritmos são soluções iterativas para o problema e, em geral, são pouco eficientes para instâncias grandes. O primeiro algoritmo que estudaremos é a ordenação pelo método de bolhas ou, mais popularmente, *Bubble Sort*.

Ordenação pelo Método de Bolhas (*Bubble Sort*)

O algoritmo de ordenação pelo método de bolhas, popularmente conhecido como *Bubble Sort*, é de simples aplicação e entendimento, porém ineficiente. Ele considera a troca repetitiva de elementos vizinhos que estão fora de ordem. Os passos considerados pelo *Bubble Sort* são descritos no Algoritmo 1.

Algoritmo 1 – Ordenação pelo método de bolhas (*Bubble Sort*)

BubbleSort(V)

1. **para** $i \leftarrow 0$ **até** $|V| - 1$
2. **para** $j \leftarrow |V| - 1$ **até** $i + 1$
3. **se** $V[j] < V[j - 1]$
4. **trocar** $V[j]$ **com** $V[j - 1]$

O *Bubble Sort* recebe como parâmetro de entrada uma sequência de valores V . Considere, a princípio, a entrada V como sendo um vetor (*array*) estaticamente alocado, ou seja, de tamanho previamente definido e armazenado sequencialmente na memória, esse tipo de estrutura será considerado para todos os algoritmos apresentados nesta unidade. Além disso, considere que $|V|$ se refere ao tamanho do vetor e define o número de elementos armazenados nessa estrutura. Nesse contexto, se $V = [1, 4, 2, 0, 3]$ então $|V| = 5$.

O laço externo, descrito na linha 1 do *Bubble Sort*, é responsável por percorrer o vetor em todas as suas posições; assim, todas as instruções que estão aninhadas a esse laço serão executadas para cada um dos elementos no vetor. Por outro lado, o próximo laço (interno), descrito na linha 2, será responsável por percorrer o vetor no sentido oposto, ou seja, da última até a segunda posição (para a primeira execução do laço externo).

Para cada elemento no vetor, é feita sua comparação com o elemento anterior (linha 3) e caso o elemento atual seja menor que o elemento anterior, é realizada a troca entre esses elementos (linha 4).

A Figura 1 ilustra cada passo do algoritmo *Bubble Sort* executado sobre o vetor de inteiros $V = [5, 4, 3, 2, 1]$. Note que V se refere ao pior caso, isto é, os elementos no vetor estão na ordem inversa daquela pretendida. Inicialmente, na linha 1 e coluna a, a variável $i = 0$ marca o ponto final até o qual será considerado para as comparações. Por outro lado, a variável $j = 4$ marca o ponto inicial das comparações. Note que $V[j] < V[j - 1]$, pois $V[4] = 1$ e $V[3] = 2$, assim os valores dessas posições devem ser trocados.

No próximo passo, ilustrado na linha 1 e coluna b, a variável j é decrementada em uma unidade, passando para $j = 3$. Nesse caso, a comparação será entre $V[3]$ e $V[2]$; como $V[3] = 1$ é menor que $V[2] = 3$, novamente, a troca deve ser feita entre essas posições.

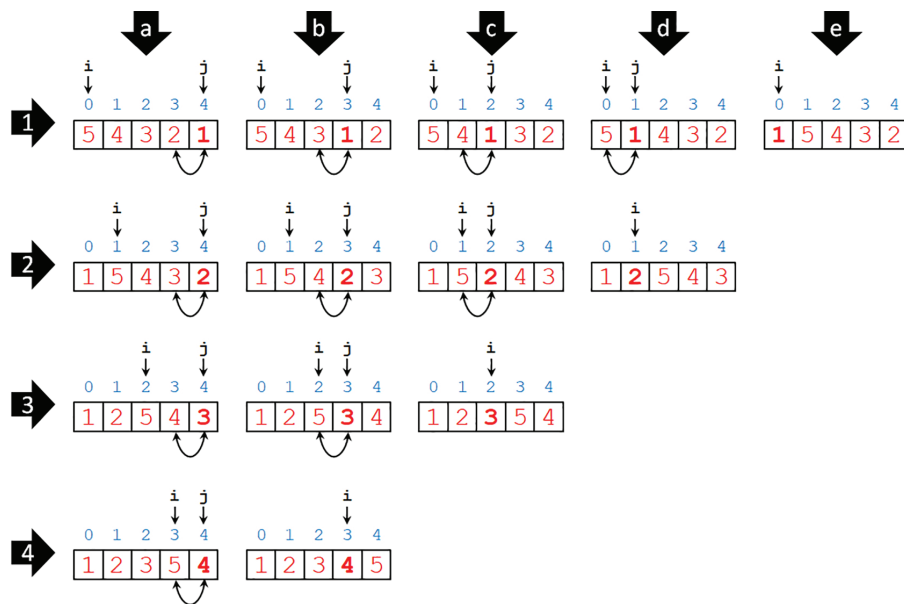


Figura 1 – Passo a passo da execução do algoritmo *Bubble Sort* sobre um vetor de inteiros

Fonte: Acervo do Conteudista

O procedimento será repetido até que tenhamos $j = 1$; veja que nesse ponto (linha 1, coluna d) a comparação será entre $V[1] = 1$ e $V[0] = 5$ e como $V[1] < V[0]$, a troca será feita. Quando a variável j for decrementada novamente, ela interromperá o laço interno (j) e o laço externo (i) deverá recomençar com o incremento da variável i . Nesse ponto da execução do algoritmo (linha 1, coluna e), o menor valor da sequência está posicionado no início do vetor. Desse modo, quando a variável i for incrementada, o vetor parcial, à esquerda de i , estará na ordem correta, não necessitando ser considerado para os próximos passos.

Após o início de um novo ciclo do laço externo, teremos $i = 1$ (linha 2, coluna a) e, conseqüentemente, o laço interno, também, será reiniciado com $j = 4$. Veja que o processo não se altera, sempre será comparado o valor de $V[j]$ com $V[j - 1]$ e, a depender do resultado da comparação, a troca será realizada ou não. Após a finalização do segundo ciclo do laço externo, controlado pela variável i , teremos o valor 2 armazenado na posição $V[1]$ (linha 2, coluna d). Assim, inicia-se o terceiro ciclo do laço externo, incrementando a variável i , o que resultará em $i = 2$ (linha 3, coluna a). Veja que o vetor parcial, à esquerda da posição $i = 2$, que é composto pelas posições $V[0]$ e $V[1]$, já está ordenado e o processamento a partir desse ponto não considerará essas posições.



Importante!

Existem diferentes versões para o algoritmo *Bubble Sort*, algumas localizam o maior valor e o levam até o final do vetor, o que é o inverso do que a nossa versão faz. Note que, independentemente da versão, o *Bubble Sort* percorre todo o vetor, identifica o menor ou o maior valor, segrega esse valor em uma das extremidades e reduz o tamanho do vetor.

O procedimento se repete até que o laço externo, controlado pela variável i , seja interrompido. Considerando a Figura 1, note que teríamos uma quinta linha na representação. Porém, nessa linha o laço externo teria $i = 4$ e o laço interno, também, teria $j = 4$, desse modo não haverá nenhuma instrução a ser executada e o algoritmo finalizará.

O algoritmo *Bubble Sort* compara todos os valores de sua entrada aos pares, trocando os valores que estão fora de ordem. Essa comparação pode ser feita do final para o início do vetor, como fizemos aqui, ou do início para o final, como é mais comumente feito. Independentemente do sentido da execução, não há diferença sobre a eficiência de ambas as versões. Ambas são ineficientes.

Agora que você compreendeu o funcionamento do algoritmo *Bubble Sort*, é hora de implementá-lo. Há diferentes implementações disponíveis na *internet* para o algoritmo *Bubble Sort*, a maioria considera as comparações feitas pelo algoritmo do início para o final do vetor. Procure uma implementação que lhe seja conveniente, considerando qualquer linguagem de programação, e tente alterá-la para a versão apresentada aqui.



O artigo a seguir, faz uma descrição do funcionamento do algoritmo e oferece diversas opções de implementações, nas mais variadas linguagens de programação.

Disponível em: <https://bit.ly/3urB7kc>

Ordenação por Inserção (*Insertion Sort*)

O algoritmo *Insertion Sort* é o mais eficiente dos algoritmos de ordenação que gastam tempo proporcional a uma função quadrática, para instâncias pequenas. Uma analogia, comumente utilizada para ajudar na compreensão do funcionamento do *Insertion Sort*, é a forma pela qual algumas pessoas organizam um conjunto de cartas de baralho na mão.

Quando jogamos algum tipo de jogo com baralho, a organização das cartas na mão é feita de forma similar ao funcionamento do *Insertion Sort*. Primeiro, imagine que você está com suas cartas empilhadas na mesa de jogo e vai pegá-las uma a uma. A cada carta retirada da pilha, compara-se com as cartas que estão na mão esquerda, até que se encontre o lugar adequado para o posicionamento da carta.

Considere o Algoritmo 2, que descreve os passos previstos para a ordenação por inserção, vamos analisá-lo de acordo com a analogia das cartas. Quando selecionamos a primeira carta na pilha, a mão esquerda está vazia; assim, simplesmente posicionamos a carta na mão. Da mesma maneira, o algoritmo utiliza um laço (linha 1) onde a variável j é inicializada com o valor 1. Nesse caso, o laço representará as cartas retiradas da pilha,

a partir da segunda carta, visto que a primeira carta é representada pelo valor 0, ou a primeira posição no vetor. Assim, na primeira execução do laço, a segunda carta ($V[1]$) é comparada com a primeira ($V[0]$); note que o valor que será objeto de comparação é reservado na variável *chave*.

Nesse contexto, o valor reservado, que é identificado pela variável *j*, será comparado com os valores que estão posicionados à esquerda de *j*. A comparação verifica, no laço interno (linha 4), se os valores referenciados pela variável *i* são maiores que o valor reservado na variável *chave*. Enquanto a comparação resultar no valor lógico verdadeiro, os valores são deslocados uma posição em direção ao final do vetor (linha 5). Ao final da execução do laço interno, o valor na *chave* é posicionado de acordo com a indicação da variável *i*. O procedimento descrito será executado para todos os valores no vetor.

Algoritmo 2 – Ordenação por inserção (*Insertion Sort*)

InsertionSort(*V*)

1. **para** $j \leftarrow 1$ até $|V| - 1$
2. $chave \leftarrow V[j]$
3. $i \leftarrow j - 1$
4. **enquanto** $i \geq 0$ e $V[i] > chave$
5. $V[i+1] \leftarrow V[i]$
6. $i \leftarrow i - 1$
7. $V[i+1] \leftarrow chave$

A Figura 2 apresenta os passos do *Insertion Sort* sobre um vetor de inteiros com cinco elementos. Veja que, da mesma forma considerada para o algoritmo *Bubble Sort*, o vetor está na ordem inversa que se pretende, ou seja, o vetor representa o pior caso para a ordenação. Também de maneira similar àquela considerada para simulação do *Bubble Sort*, as linhas numeradas, na Figura 2, representam os incrementos da variável *j* e as colunas, identificadas com letras, representam os incrementos da variável *i*.

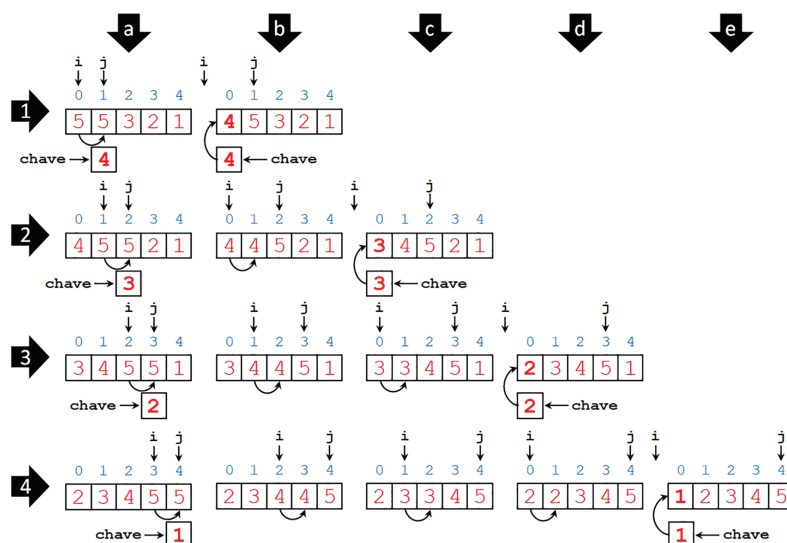


Figura 2 – Passo a passo da execução do algoritmo *Insertion Sort* sobre um vetor de inteiros

Fonte: Acervo do Conteudista

Inicialmente, na linha 1 e coluna a, a variável j recebe o valor 1, essa atribuição identifica a posição do valor que será comparado ($V[1]$), o qual será atribuído à variável *chave*, e a variável i recebe $j - 1$, que, nesse momento, será igual a 0. Veja que $V[i] = V[0] = 5$, que é um valor maior que aquele armazenado na variável *chave*; assim, o algoritmo prevê o deslocamento do valor 5 uma posição em direção ao final do vetor, ou seja, ele é atribuído à posição $V[i + 1] = V[1]$.

O próximo passo é o decremento da variável i , que passa a armazenar -1 . Como $i = -1$, o laço interno é interrompido, pois $i \geq 0$ é falso, e o conteúdo da variável *chave* é atribuído de volta ao vetor na posição $V[i + 1] = V[-1 + 1] = V[0]$. O estado do vetor, após a primeira execução do laço externo, é $V = [4, 5, 3, 2, 1]$.

A linha 2, na Figura 2, representa a segunda execução do laço externo, agora com o incremento da variável j , ou seja, $j = 2$. Da mesma forma vista no passo anterior, a variável *chave* recebe o valor da posição j , isto é, $chave = V[j] = V[2] = 3$. O valor reservado na *chave* será comparado com os valores à esquerda de sua posição original e, caso esses valores sejam maiores que a *chave*, eles são deslocados uma posição à direita. Ao final das comparações, o valor reservado na *chave* retorna ao vetor na posição correta.

Veja que o padrão se repete a cada iteração de cada um dos laços. O algoritmo seleciona o próximo valor no vetor, reserva esse valor, realiza a comparação com os valores à esquerda; se o valor for maior que o valor reservado, ele é deslocado para a direita e, finalmente, o valor reservado é posicionado de forma correta. O algoritmo finaliza sua execução após todos os valores terem sido verificados. Na linha 4 e coluna e, na Figura 2, temos a finalização de mais uma execução do laço interno, governado pela variável i , e o laço externo incrementa a variável j para mais uma execução. Porém, nesse ponto, j será $j = 4 + 1 = 5$, o que excede o limite estabelecido por $|V| - 1 = 5 - 1 = 4$ e o algoritmo finaliza sua execução.



A simulação do funcionamento dos algoritmos é a melhor forma de compreender seu comportamento. Considere um vetor que não esteja no pior caso, por exemplo: $V = [5, 2, 3, 1, 4]$, e faça a simulação do *Insertion Sort*, da mesma forma apresentada na Figura 2.

Na próxima seção, vamos conhecer o último algoritmo de ordenação dessa série inicial que estamos abordando, trata-se do algoritmo *Selection Sort*. Nesse grupo inicial, os algoritmos de ordenação selecionados têm em comum o fato de considerarem uma abordagem iterativa em suas definições, em vez de uma abordagem recursiva. Nas unidades futuras, trataremos dos algoritmos que utilizam a abordagem recursiva. Outra característica comum a esses algoritmos é que todos são ineficientes para grandes instâncias de entrada, ou seja, quando precisamos ordenar muitos elementos, esse algoritmos não são nossa melhor escolha.

A comparação entre diferentes algoritmos de ordenação possibilita a compreensão de estratégias distintas, que certamente serão úteis em aprimorar nossa capacidade de desenvolver nossas próprias soluções.

Ordenação por Seleção (*Selection Sort*)

O algoritmo *Selection Sort* é um dos mais eficientes para a ordenação de pequenos conjuntos de elementos. Outra característica importante é o fato de o *Selection Sort* não utilizar uma estrutura auxiliar para o armazenamento dos dados do vetor e, dessa forma, consome pouca memória em seu processo de execução. O Algoritmo 3 descreve os passos pertinentes ao *Selection Sort*.

Algoritmo 3 – Ordenação por seleção (*Selection Sort*)

SelectionSort(V)

1. **para** $i \leftarrow 0$ **até** $|V| - 2$
2. $min \leftarrow i$
3. **para** $j \leftarrow i + 1$ **até** $|V| - 1$
4. **se** $V[j] < V[min]$
5. $min \leftarrow j$
6. **se** $V[i] \neq V[min]$
7. trocar $V[i]$ com $V[min]$

O algoritmo *Selection Sort* considera dois laços, um externo, indexado pela variável i , e outro interno, indexado pela variável j . O laço externo é responsável por selecionar do primeiro ao penúltimo elemento do vetor e, a cada iteração desse laço, o menor elemento será armazenado na posição $V[j]$. O laço interno cuida da comparação de cada elemento, selecionado pelo laço externo, com todo o vetor parcial direito, ou seja, os elementos que estão à direita de $V[i]$.

Na primeira iteração do laço externo, o algoritmo assume que a posição indexada pela variável i contém o menor elemento do vetor. Deste modo, em vez de o algoritmo reservar o valor armazenado no vetor, ele reserva, na variável min , a sua posição, ou seja, o valor do indexador i . O laço interno compara todos os elementos à direita de $V[i]$ com o valor que está indexado e, caso o valor atual ($V[j]$) seja menor que o valor reservado pelo indexador ($V[min]$), o algoritmo atualiza o valor da variável min com o valor de j .

Ao final da execução do laço interno, no caso de o valor apontado por i ser diferente do valor apontado por min , o algoritmo realiza a troca dos valores que estão nessas posições. Nesse ponto, teremos o menor valor do vetor armazenado na posição i e, após o incremento de i , inicia-se uma nova execução do laço externo. O algoritmo termina quando $i > |V| - 2$, ou seja, após considerar o penúltimo elemento da série.

Considere a Figura 3, na qual as linhas referem-se às execuções do laço externo e as colunas representam as execuções do laço interno.

Na linha 1 e coluna a, temos a representação do vetor antes do início da execução do algoritmo *Selection Sort*. Vamos considerar o vetor em duas partes, um vetor parcial esquerdo ordenado e um vetor parcial direito desordenado. Inicialmente, temos somente o vetor desordenado ($V[1, \dots, 4]$). Veja que a variável $i = 0$ marca a posição que será

reservada na variável min como sendo aquela que, por enquanto, armazena o menor valor. Ainda na mesma representação, a variável j marca o início do vetor parcial direito desordenado (vamos chamar apenas de vetor direito daqui em diante). A primeira comparação entre $V[min]$ e $V[j]$ indica que o valor armazenado na posição j é menor que o valor armazenado na posição min . Assim, a variável min é atualizada com o valor de j .

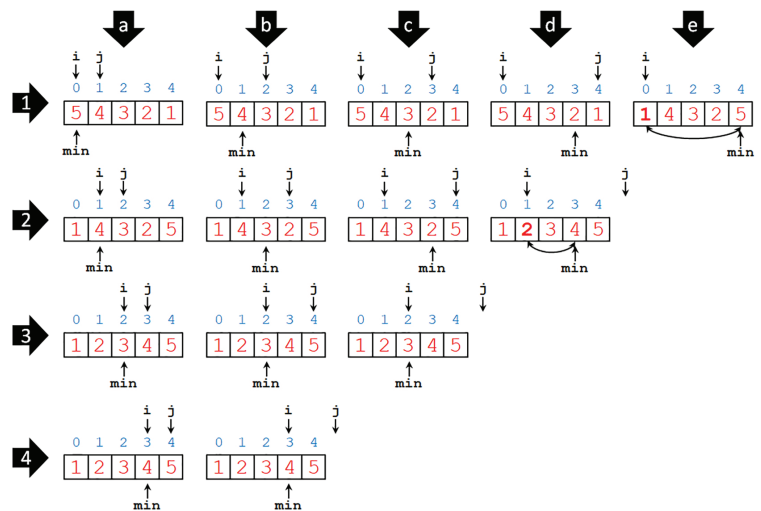


Figura 3 – Passo a passo da execução do algoritmo *Selection Sort* sobre um vetor de inteiros

Fonte: Acervo do Conteudista

O processo descrito se repete sempre da mesma maneira, a variável j percorre as posições do vetor direito, comparando os valores armazenados com o valor armazenado na posição min . Quando encontra um valor menor que o apontado por min , o algoritmo atualiza o valor de min , de forma a apontar para o menor valor identificado até aquele momento.

O laço externo do algoritmo finaliza sua execução quando j percorrer todas as posições no vetor. Quando esse laço é finalizado, o algoritmo verifica se os valores armazenados em $V[i]$ e $V[min]$ são diferentes e, caso sejam, ele troca os valores dessas posições. Veja que na Figura 3, linha 1, coluna e, o algoritmo identificou o valor 1 (posição 4) como o menor do vetor direito e trocou de posição com o valor em i , que é igual a 5 (posição 0).

A segunda iteração do laço externo, representado na Figura 3 pela linha 2, inicia com o incremento da variável $i = 1$. Veja que essa variável indica o ponto que divide o vetor esquerdo ($V[0]$) do direito ($V[2, 3, 4]$). Da mesma forma observada na iteração anterior, o algoritmo assume que a posição indicada por i contém o menor valor e reserva essa posição na variável min . As posições do vetor direito são percorridas e comparadas individualmente com o valor indicado por min . Caso uma nova posição contenha um valor menor, a variável min é atualizada. Veja que, quando $j = 4$ o valor referente a essa posição não é menor que o valor apontado pela variável min , assim ela não é atualizada (linha 2, coluna c).

Quando o laço externo finaliza, observamos que $V[i = 1] = 4$ e $V[min = 3] = 2$; desse modo, os valores dessas posições são trocados, conforme prevê o algoritmo. Note que essa última troca já produz a ordenação de todos os elementos no vetor, porém o algoritmo segue sua execução da mesma maneira, mas sem produzir nenhuma troca de valores no vetor.

O estudo dos algoritmos de ordenação possibilita uma visão mais ampla sobre diferentes estratégias. Quando analisamos os algoritmos *Bubble Sort*, *Insertion Sort* e *Selection Sort*, vemos que eles consideram diferentes estratégias e que cada uma delas pode ter mais ou menos operações.

As operações, também, consomem tempos diferentes para serem realizadas. Por exemplo, uma comparação entre dois valores pode ser mais rápida de ser executada que a atribuição de um valor a uma variável. Ou ainda, uma simples troca de valores em um vetor demora mais que uma atribuição, visto que para a troca é necessário fazer três atribuições.

Nesse contexto, é possível verificar que podemos ter algoritmos que realizam a mesma tarefa, como é o caso dos algoritmos de ordenação, mas fazem isso de maneiras diferentes. Essas diferenças entre as estratégias, consideradas pelos algoritmos, impactam no tempo que será necessário para a execução da tarefa.

Para ilustrar os diferentes desempenhos dos algoritmos de ordenação considerados até o momento, Souza *et al.* (2017) realizou um experimento com o objetivo de observar as diferenças entre os algoritmos, com relação ao tempo de execução, os números de comparações e de trocas realizadas. Foi considerado um vetor com 100 elementos, ordenados de formas crescente, decrescente e aleatória.

Veja que as diferentes organizações do vetor impacta diretamente no tempo que o algoritmo vai demorar em realizar a ordenação. O primeiro caso considera um vetor em ordem crescente, isso é o que chamamos de melhor caso. Para o melhor caso não serão realizadas trocas, em nenhum dos algoritmos, visto que os valores já estão em suas posições corretas, de acordo com o objetivo do algoritmo. Há somente a realização de comparações que verificam as respectivas posições dos valores no vetor.

Quando o vetor tem seus valores organizados em ordem decrescente, dizemos que temos o pior caso. Nesse contexto, o pior caso é quando todos os valores serão reposicionados, considerando um vetor com um número par de elementos.

O último caso, conhecido como caso médio, é quando os valores estão posicionados de forma aleatória no vetor. Esse é o caso mais comum de ser observado.

Tabela 1– Comparação de desempenho entre os algoritmos de ordenação para diferentes casos

	Crescente			Decrescente			Aleatória		
	Tempo	Comp.	Trocas	Tempo	Comp.	Trocas	Tempo	Comp.	Trocas
<i>Bubble Sort</i>	0,00024	4950	0	0,00027	4950	4950	0,00024	4950	2377
<i>Insertion Sort</i>	0,00015	99	0	0,00020	99	4950	0,00018	99	24941
<i>Selection Sort</i>	0,00020	4950	0	0,00019	4950	50	0,00020	4950	93,5

Fonte: SOUZA *et al.*, 2017

A Tabela 1 apresenta um quadro comparativo sobre os diferentes desempenhos dos algoritmos de ordenação considerados. Nesse contexto, foram analisados o tempo de execução, o número de comparações e a quantidade de trocas realizadas pelos algoritmos *Bubble Sort*, *Insertion Sort* e *Selection Sort*. Além disso, foram submetidas diferentes

instâncias de entrada, as quais apresentavam ordenações prévias crescente, decrescente e aleatória. Veja que os resultados apresentados na Tabela 1 são valores médios.

Uma análise superficial dos dados na tabela revela que os desempenhos dos algoritmos variam em função da estratégia considerada e, também, da organização prévia dos elementos no vetor. Desse modo, podemos concluir que, de fato, os algoritmos que realizam uma mesma tarefa podem ter desempenhos diferentes, o que, para grandes instâncias de entrada, pode inviabilizar seu uso.

Pesquisa Sequencial

O processo de pesquisa ou busca de um determinado valor em uma lista é uma operação cotidiana, no âmbito da computação. Existem, basicamente, duas formas de se realizar uma pesquisa. Nesta unidade, trataremos da forma mais simples e ineficiente de pesquisa chamada de pesquisa sequencial.

A pesquisa sequencial considera um valor x a ser procurado e uma lista de valores V onde será realizada a pesquisa. Veja que, para o contexto desta unidade, utilizaremos um vetor estático para realizar o processo de pesquisa.

O Algoritmo 4 descreve a função *Busca()*, que retorna um valor lógico (verdadeiro ou falso), indicando a existência ou não do valor no respectivo vetor. A função recebe como parâmetros de entrada o valor a ser pesquisado e o vetor de elementos.

Algoritmo 4 – Algoritmo para pesquisa sequencial

Busca(x, V)

1. **para** $i \leftarrow 0$ **até** $|V|$
2. **se** $x = V[i]$
3. **retorna** VERDADEIRO
4. **retorna** FALSO

A função *Busca()* tem um único laço que é responsável por percorrer todo o vetor, considerando a ordem em que os valores estão armazenados. Para cada elemento no vetor, a função faz uma comparação entre o valor objetivo (x) e o elemento correspondente. Caso haja a igualdade entre os valores, a função retorna o valor lógico verdadeiro e encerra a busca; caso contrário, retorna falso. Veja que a função somente retornará falso quando todo o vetor for percorrido e o valor objetivo não for encontrado; desse modo, o valor lógico falso indica que o valor objetivo não pertence ao vetor de elementos.

A ideia central na função *Busca()* é bastante simples e pode ser considerada para aplicações mais práticas. Considere, por exemplo, o armazenamento de diferentes valores para um mesmo registro e diversos registros armazenados em um vetor. Nesse caso, passamos de um vetor unidimensional para um com duas dimensões. Um vetor bidimensional

é, também, conhecido como matriz; assim, uma matriz é um vetor no qual os elementos armazenados são, também, vetores.

Considere a Figura 4, veja que temos um vetor principal com n registros. Cada um dos registros é um vetor que contém os dados referentes a uma pessoa, ou seja, o CPF, o nome, o telefone e a idade. Podemos interpretar a figura de outra forma, como uma matriz bidimensional com n linhas e quatro colunas. Assim, cada linha é um registro dos dados da pessoa e cada coluna contém um valor específico para aquele atributo. A forma que se considere para interpretar a estrutura não tem impacto direto na sua utilização.

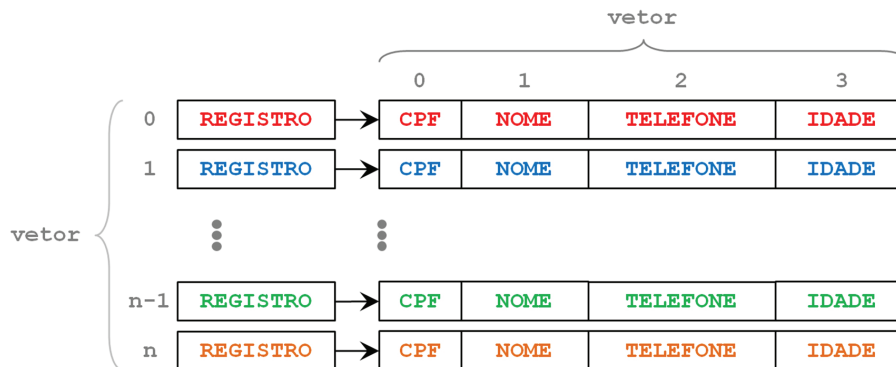


Figura 4 – Exemplo de uma matriz abstraída na forma de dois vetores

Fonte: Acervo do Conteudista

Podemos utilizar a função *Busca()* para retornar um registro específico de uma pessoa, alterando poucas instruções. Os parâmetros de entrada continuam sendo o vetor, ou mais especificamente a matriz de dados, e o número do CPF da pessoa. Veja que este atributo é exclusivo e, assim, podemos localizar exatamente o registro que se pretende.

A comparação que é realizada pela função *Busca()* deverá ser alterada, de modo que o valor do CPF, representado pela variável x , seja comparado da seguinte forma:

$$sex = V[i][0]$$

Veja que a variável i continua sendo responsável por percorrer todos os registros e, para cada registro, a comparação é realizada considerando sua primeira posição, a qual contém o valor do CPF e está indexada pelo valor 0.

Outra alteração necessária de se realizar na função *Busca()* é o seu retorno. No lugar de um valor lógico, a função retornará um vetor ($V[i]$) com os dados da pessoa. Na verdade, considerando a implementação da função, o valor retornado por ela será o endereço físico de memória daquele vetor específico e não os valores de fato.

Vamos ver outro exemplo, para a função, utilizando uma sentinela. Para o contexto desta unidade, onde estamos tratando exclusivamente de alocação estática com a utilização de vetores, o exemplo com sentinela necessita de um cuidado prévio. O vetor que receberá os dados deve ter uma posição adicional. Assim, caso tenhamos dez valores a serem armazenados, devemos alocar 11 posições. A posição adicional servirá para que possamos inserir o valor procurado no vetor. Veja no Algoritmo 5 o exemplo da função considerando a sentinela.

Algoritmo 5 – Algoritmo para pesquisa sequencial com a utilização de sentinela

BuscaComSentinela(x, V)

1. $i \leftarrow 0$
2. $V[|V|-1] \leftarrow x$
3. **enquanto** $x \neq V[i]$
4. $i \leftarrow i + 1$
5. **se** $i = |V| - 1$
6. **retorna** *FALSO*
7. **senão**
8. **retorna** *VERDADEIRO*

A função *BuscaComSentinela()* adiciona o valor procurado x na posição final do vetor, lembrando que devemos prever essa posição adicional no vetor. O laço incrementa a variável i em uma unidade enquanto o valor que estamos buscando não for encontrado. Há duas possibilidades de finalização do laço. Na primeira, o valor existe no vetor e, por óbvio, está localizado antes do valor sentinela, que ocupa a última posição. Assim, o valor armazenado na variável i será menor que o valor da última posição do vetor ($|V| - 1$) e a função retornará o valor lógico verdadeiro, indicando que o valor buscado está presente no vetor.

A segunda possibilidade de finalização do laço é quando ele chega até a sentinela. Nesse caso, o laço percorrerá todos os elementos até a última posição, onde está a sentinela que interromperá o ciclo do laço. O valor da variável i quando o laço for interrompido será igual ao tamanho do vetor menos um ($|V| - 1$) e a função retornará falso, indicando que o valor procurado não foi encontrado no vetor.

A busca sequencial apresenta uma característica que pode tornar esse processo muito ineficiente. Veja que todas as posições do vetor serão percorridas na ordem em que aparecem. Para o caso de o valor procurado estar na última posição, ou não existir no vetor, a função *Busca()* fará $n = |V|$ comparações. Supondo um vetor com um milhão de elementos, serão um milhão de comparações.

Uma alternativa à busca sequencial é a busca binária. Na busca binária, o número de comparações realizadas é muito menor, tornando o processo de pesquisa muito mais eficiente. Porém é necessário que o vetor esteja ordenado previamente para se executar a busca binária. Veremos nas próximas unidades os detalhes referentes a esse tipo de busca.



Antecipe-se sobre o assunto pesquisa binária, procure saber as principais características dessa abordagem e as diferenças para a pesquisa sequencial.

Nesta unidade estudamos diferentes algoritmos de ordenação. Esses algoritmos têm em comum a utilização de uma estratégia iterativa de ordenação. Os algoritmos iterativos são aqueles que repetem um conjunto de instruções diversas vezes. Veja que os

algoritmos considerados até agora utilizam laços de repetição, por vezes aninhados, em sua estratégia de ordenação. A alternativa seria a utilização da recursão como estratégia de ordenação. Nas próximas unidades, veremos outros algoritmos de ordenação que consideram a recursão como estratégia.

Outro ponto em comum entre os algoritmos *Bubble Sort*, *Selection Sort* e *Insertion Sort* são as respectivas eficiências computacionais. Por eficiência computacional considere o tempo que o algoritmo demora em produzir a saída correspondente. Esses algoritmos são úteis quando temos poucos elementos a serem ordenados. Outra possibilidade é a utilização de algum desses algoritmos associados a outros algoritmos mais eficientes. Nesse contexto, os algoritmos mais eficientes, também, serão apresentados nas próximas unidades.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Vídeos

Bubble-sort with Hungarian ("Csángó") folk dance

<https://youtu.be/lyZQPjUT5B4>

Insert-sort with Romanian folk dance

<https://youtu.be/ROaIU379I3U>

Select-sort with Gypsy folk dance

<https://youtu.be/Ns4TPTC8whw>

Linear search with Flamenco dance

<https://youtu.be/-PuqKbu9K3U>

Referências

CORMEN, T. H. *et al.* **Introduction to algorithms**. MIT press, 2009.

SOUZA, J. E. G.; RICARTE, J. V. G.; LIMA, N. C. A. Algoritmos de ordenação: um estudo comparativo. **Anais do Encontro de Computação do Oeste Potiguar ECOP/UFERSA** (ISSN 2526-7574), n. 1, 2017.

SZWARCFITER, J. L. **Estruturas de dados e seus algoritmos**. 3ª ed. Rio de Janeiro: LTC, 2010. (*e-book*)



Cruzeiro do Sul
Educatonal